Legacy Systems and Software Reengineering

DASS - Week 14

Software Aging

"Software is a mathematical product; mathematics doesn't decay with time. If a theorem was correct 200 years ago, it will be correct tomorrow. If a program is correct today, it will be correct 100 years from now. If it is wrong 100 years from now, it must have been wrong when it was written. It makes no sense to talk about software aging."

60-80%

of enterprise IT budget spent on maintaining existing systems — not new development
(Forrester Research)

Four Types of Software Changes

Change Type	Code Structure	Functionality	Resource Usage	Architectural Impact
Adding a Feature	**Changes**	**Changes**	-	*May require new interfaces or components*
Fixing a Bug	**Changes**	**Changes**	-	*Should be localised; if not, design is poor*
Refactoring	**Changes**	Unchanged	-	*Intent: improve structure without altering behaviour*
Optimising	-	Unchanged	**Changes**	*Performance tuning; correctness must not change*

Smells as Indicators…

Long Method

A function/method that does far too many things.

Fix: Extract smaller, named methods from the body.

Large Class (God Object)

One class knows and controls everything.

Fix: Split by responsibility (Single Responsibility Principle).

Duplicate Code

The same or similar logic appears in multiple places.

Fix: Extract to a shared function, class, or module.

Feature Envy

A method more interested in another class's data than its own.

Fix: Move the method to the class whose data it uses.

Dead Code

Unreachable or unused code that accumulates over years.

Fix: Delete it. Version control preserves history.

Comments as Deodorant

Extensive comments needed to explain what the code does.

Fix: Refactor the code to be self-explanatory.

"Edit and Pray" vs "Cover and Modify"

Edit and Pray (avoid this)

- Locate the area to change
- Make the change directly in the production code
- Pray it works / test manually (if at all)

Risks:

- Introduces silent regressions that surface in production
- No safety net to detect breakage
- Each change increases fear — developers become reluctant to change anything
- Common outcome: "if it ain't broke, don't touch it" paralysis. The system cannot evolve.

Cover and Modify (preferred)

- Understand the code that needs to change
- Write tests that document existing behaviour
- Introduce the change with tests in place
- Verify all tests still pass

Benefits:

- Changes can be made confidently; regressions are caught fast
- Tests become permanent documentation of expected behavior. Tests capture "what the code does now", not what it should do in future.

What Is Software Maintenance?

ISO/IEC/IEEE 14764:2022 — *"The modification of a software product after delivery to correct faults, to improve performance or other attributes, or to adapt the product to a modified environment."*

Standards History

IEEE Std 729-1983

- Original definition; now superseded

IEEE 1219-1998

- Replaced 729; added process guidance

ISO/IEC 14764:2006

- International standard; widely adopted

ISO/IEC/IEEE 14764:2022

- Current active standard

Key Characteristics

- **Begins after the software is delivered**
- Does NOT normally involve major architectural redesign (for routine maintenance)
- Changes are implemented by modifying existing components or adding new ones
- Maintenance is a continuous process throughout the system's entire useful lifetime

Four Types of Software Maintenance (ISO/IEC 14764)

Corrective

Reactive | ~20% of effort

Modification performed after delivery to correct discovered faults.

When:
A bug is reported by a user or detected in production.

Example:
Fixing a divide-by-zero error in a billing calculation.

Adaptive

Reactive | ~25% of effort

Modification to keep a program usable in a changed or changing environment.

When:
OS upgrade, new hardware, regulatory change, or third-party API change.

Example:
Updating database access code after migrating from Oracle to PostgreSQL.

Perfective

Proactive | ~40% of effort

Modification to improve performance, maintainability, or other attributes.

When:
Performance bottleneck identified, or code becomes too complex to work with.

Example:
Refactoring a spaghetti-code module to improve readability and speed.

Preventive

Proactive | ~15% of effort

Modification to detect and correct latent faults before they become failures.

When:
Code review reveals fragile code that will fail under expected future conditions.

Example:
Adding input validation to prevent future SQL injection vulnerabilities.

Maintenance Prediction

Complexity Metrics

- Cyclomatic complexity (McCabe):
  - Number of linearly independent paths through code. High values = more test cases and more risk.
- Coupling & cohesion:
  - High coupling between modules means change propagates widely. Low cohesion means modules do too much.
- Method/module size:
  - Large methods and classes are harder to understand and test in isolation.
- Research finding: most maintenance effort is concentrated in a small number of high-complexity components.

Process Metrics (track over time)

- Number of corrective maintenance requests per period
- Average time required for impact analysis
- Average time to implement a change request
- Number of outstanding (backlogged) change requests
- Defect density (defects per KLOC)

What is a Legacy System?

"legacy"

> *A sum of money, or a specified article, given to another by will; anything handed down by an ancestor or predecessor. — Oxford English Dictionary*

- A legacy system is a piece of software that
  - You have ***inherited*** and
  - Is ***valuable*** to you

- Typical problems with legacy systems:
  - original developers ***not available***
  - ***outdated*** development methods used
  - extensive patches and ***modifications*** have been made
  - ***missing*** or outdated documentation

Real-World Legacy Systems

COBOL in Banking

95% of ATM transactions and 43% of banking systems are built on COBOL (IBM, 2017). The language dates from 1959 yet processes over $3 trillion per day.

US Air Traffic Control

The FAA's HOST computer ran COBOL/Jovial code from the 1960s until its replacement program in the 2010s, handling thousands of flights daily.

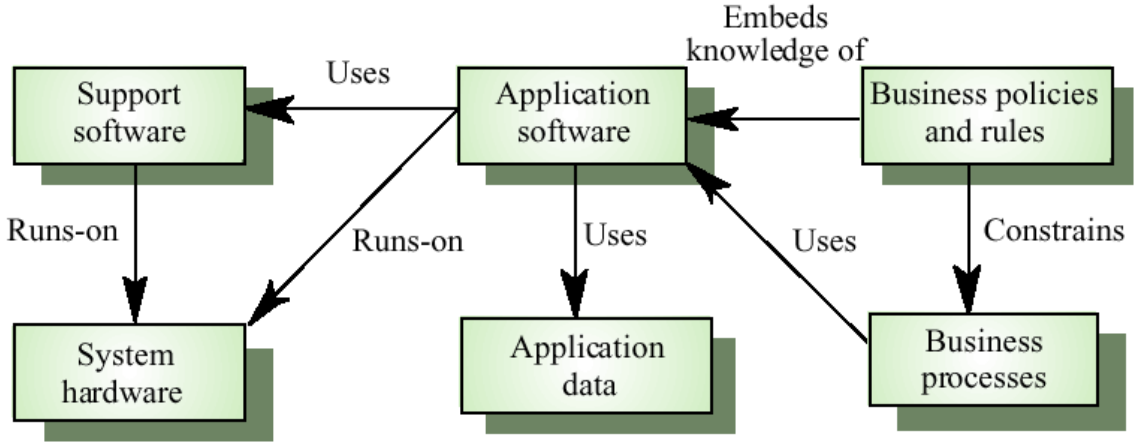
Social Security Administration

The SSA maintains ~60 million lines of COBOL, processing over $1 trillion in payments annually. Rewriting it was deemed "too risky to attempt."

Healthcare Systems

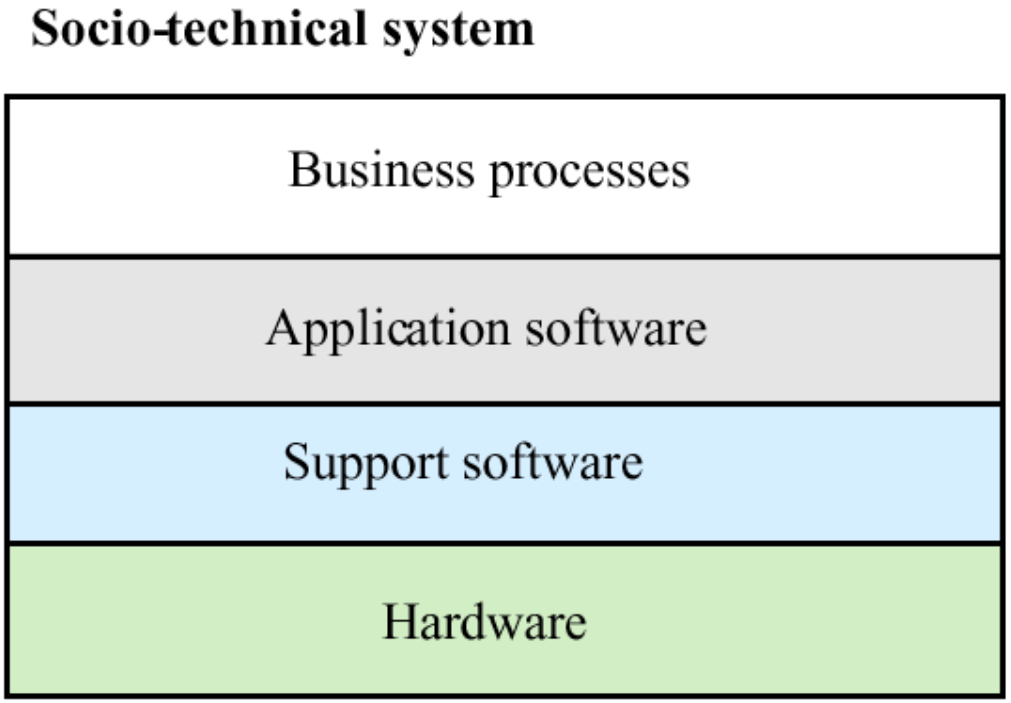
Many hospitals still run early-1990s patient management software. The HL7 standard evolved to bridge these old systems with modern EHRs

Legacy system structures



- System hardware – Mostly mainframes
- Support software – OS/Compiler support, etc
- Application software – Business services
- Application data – Normally large amount
- Business processes – Processes used in the business
- Business policies and rules – Constraints and rules

Layered model



Legacy System Design — Function-Oriented Roots

Function-Oriented Design (Legacy)

- Organised around functions/procedures, not objects
- Global data shared across many procedures
- No clear encapsulation or information hiding
- Difficult to isolate individual components for testing or replacement

Object-Oriented Design (Modern)

- Organised around objects with encapsulated state and behaviour
- Explicitly defined interfaces between components
- Supports localised change — modify one class without cascading
- Natural support for unit testing and dependency injection

Two dominant patterns in legacy function-oriented systems:

- Batch processing: data input/output in bulk from files (payroll, billing, overnight reports).
- Transaction processing: interactive, per-transaction database updates (banking reservations, order entry).

Why Legacy Systems Are Expensive to Change

Multiple Team Origins

No consistent programming style; each team made local decisions with no shared standards.

Obsolete Languages

COBOL, Fortran, PL/1 — hard to recruit, expensive to outsource.

No or Outdated Docs

The source code IS the documentation; understanding requires deep reading.

Ad Hoc Maintenance

Years of quick fixes degrade the original structure, making further changes riskier.

Optimised for Old Hardware

Tricks for memory/speed obscure logic; modern hardware assumptions no longer apply.

Data Duplication & Quality

Data spread across systems, often inconsistent; changes must propagate everywhere.

Assessing Legacy Systems

Legacy System Assessment Framework

Two dimensions must be independently assessed by interviewing multiple stakeholder groups:

Dimension 1 — Business Value

- How critical is this system to current business operations?
- What would it cost (in time, money, risk) to operate without it?
- Does it support future strategic goals?
- Could the business process be redesigned to eliminate the need?
- *Stakeholders: end-users, customers, line managers, IT managers, senior management*

Dimension 2 — System Quality

- How well does the system's supporting infrastructure perform?
- How mature and stable is the development environment?
- What is the technical quality of the application code?
- Are there adequate tests to support future changes safely?
- *Measured through: code metrics, defect history, environment assessment*

System Quality Assessment

Three quality dimensions are assessed independently. Each is scored. The aggregate score drives the strategy decision.

Business Process Quality

Key Questions:

- Is there a defined process model, and is it followed?
- Do different departments use different processes for the same function?
- Has the process been adapted informally in ways not reflected in documentation?

Low: process is ad hoc, inconsistently applied, or no longer aligned with strategy.

Environment Quality

Key Questions:

- How old and well-supported is the hardware and OS?
- Are vendor-supplied components still under active support?
- Is the development environment still commercially available and productive?

Low: obsolete hardware, end-of-life OS/DBMS, no vendor support available.

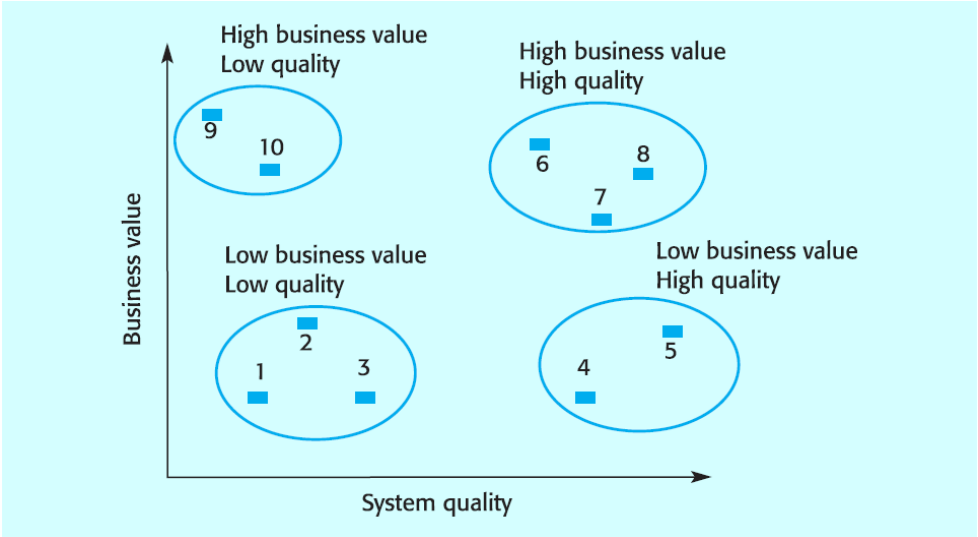
Application Quality

Key Questions:

- Is the application structured and documented well enough to change?
- What does the defect density and change request history tell us?
- Are there adequate tests? Is the code understandable by current staff?

Low: high defect density, no tests, opaque spaghetti code, frequent regressions.

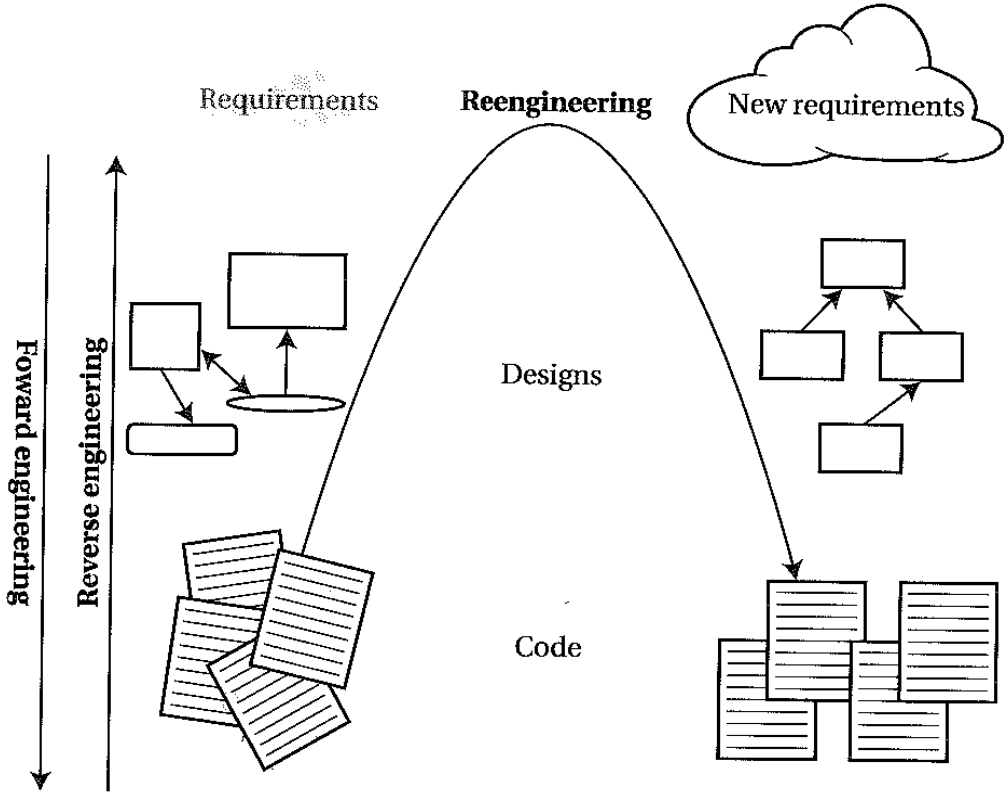
Legacy system assessment: System quality and business value



- ***Low quality, low business value*** - These are candidates for scrapping.
- ***Low quality, high business value*** - Cannot be scrapped. However, these are candidates for system transformation or replacement if a suitable system is available.
- ***High quality, low business value*** - Normal system maintenance may be continued or they may be scrapped.
- ***High quality, high business value*** - No need to invest in transformation or system replacement. Normal system maintenance should be continued.

Software Reengineering

Forward, Reverse, Re-engineering



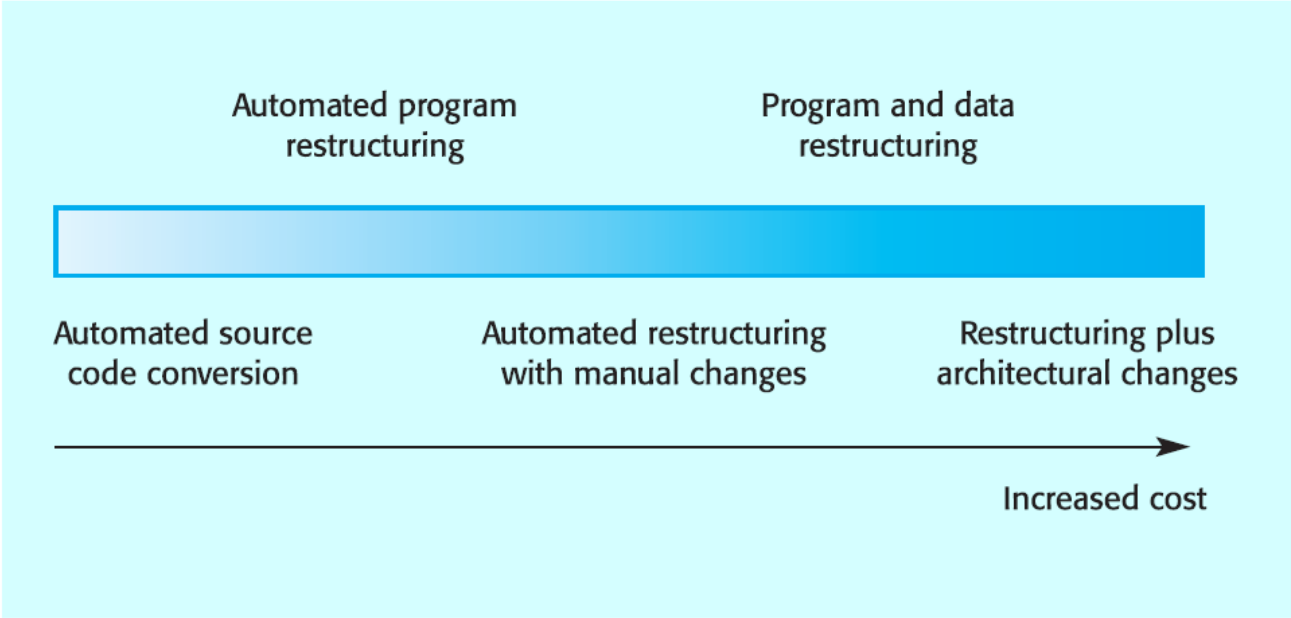
Why do we Reengineer?

- What is Legacy Software?
  - It may not be that old
- Goal of Reengineering is to reduce the complexity of legacy system sufficiently that it can be continue to be used and adapted at an acceptable cost
- Why Reengineer?
  - Unbundle a monolithic system
  - Might need an improvement in performance
  - Port the system to a new platform/technology
  - Extract the design – enables a new implementation
  - Reduce human dependencies

Warning Signs — When Reengineering Is Needed

- Obsolete or no documentation — only the code tells the story
- No automated tests — every change is a leap of faith
- Original developers and users have left; institutional knowledge is gone
- "Tribal knowledge" — only 1–2 people understand any given part of the system
- Nobody understands the full system — each person knows their silo only
- Simple changes take disproportionately long (days for a one-line fix)
- Constant stream of regressions — fixing one bug creates three others
- Long build times (hours or more) that slow all development
- Difficulties separating products or extracting features for reuse
- Widespread code duplication — the same logic appears in many places
- "Code smells" throughout — indicators of poor structural health (see next slide)

Reengineering approaches



Reengineering Cost Factors

1. Quality of the Existing Software

Favourable: Code is well-structured, has tests, and has some documentation.

Unfavourable: No tests, spaghetti code, undocumented global state, proprietary binary-only components.

Poor quality means more effort in the reverse engineering and restructuring phases — the bulk of a project's budget.

2. Tool Support Available

Favourable: Good automated migration tools (e.g., for COBOL, FORTRAN, Pascal).

Unfavourable: Bespoke proprietary language with no tooling — entirely manual.

Tools can compress the translation phase from years to months. The absence of tools shifts all cost to skilled labour.

3. Data Conversion Complexity

Favourable: Well-documented, normalised DBMS data with clean referential integrity.

Unfavourable: Binary flat files, undocumented formats, decades of inconsistency.

Data reengineering is often 40–60% of total project cost. Underestimating this is the most common cause of budget overrun.

4. Expert Staff Availability

Favourable: Original developers available for consultation; knowledge transfer possible.

Unfavourable: All original developers gone; technology completely obsolete — no market for expertise.

Scarcity of COBOL/FORTRAN expertise inflates day rates. Key-person risk also applies during the reengineering project itself.

A structured change process for aging systems

This five-step process (Feathers, 2004) applies whether adding a feature, fixing a bug, or beginning reengineering:

1

Identify Change Points

Determine exactly where in the codebase the change must be made. Resist the urge to "just start changing things" — understand the scope first.

2

Find Test Points

Identify which parts of the code need to be tested to verify the change. Test points may not be near the change point — dependencies can be surprising.

3

Break Dependencies

Legacy code is typically highly coupled. Use dependency-breaking techniques (interfaces, seams, mocks) to isolate the component you need to test.

4

Write Tests

Write characterisation tests to document current behaviour before making changes. Then write new tests for the desired new behaviour.

5

Make Changes & Refactor

Make the change incrementally, keeping tests green throughout. Use this as an opportunity to improve code structure (refactor) — leave the code cleaner than you found it.